

## Team Contribution Summary

| Team Member | Role                                      | Deliverable                        |
|-------------|-------------------------------------------|------------------------------------|
| Kate        | Data Preprocessing & Feature Engineering  | Clean Data & Engineered Features   |
| Alexia      | Model Training & Hyperparameter Selection | Optimized Model Objects            |
| Cherluk     | Validation, Metrics & Visualization       | Statistical Tables & Visual Charts |
| Jade        | Ensemble Design & Documentation           | Ensemble Model & Final Reports     |

### **I. Data Preprocessing & Feature Engineering**

**Lead: Kate Leeman**

The data preparation phase of the machine learning pipeline was used to establish integrity of the machine learning pipeline. The steps followed were as follows in order to ensure maximum performance of the model:

Preparation of Data Systems: Before the modeling process could be performed, end-to-end audit of the hotel booking data was done to ensure that the structural consistency and data quality of all the 25 raw features were consistent.

Stratified Data Partitioning: A 70/15/15 train-validation-test split was used with `sklearn.model_selection`. To ensure that the balance of the classes of the target variable (cancellations) was not skewed in both training and testing, `stratify=y` parameter was utilized to ensure that the balance of the classes of the target variable (cancellations) was not skewed in both training and testing.

Data Leakage Mitigation: This is a strict feature audit, which initially identified and eliminated reservation status and reservation status date. These columns were named causes of data leakage since they are sources of information that is not completed until the target event has occurred. Also, the low-utility fields, like agent, company, and granular date fields, were pruned to reduce noise.

Feature Engineering and Design: There are four specialized features that were created to store intricate behavioral patterns in guest data:

total\_nights: The weekend and weekday stays summed up to give the total stay.

total-guests: The number of adults, children, infants in order to quantify the scale of booking.

room changed: A flag that there was a difference between the types of rooms booked and allocated and was used as a proxy measure of guest dissatisfaction.

has children: A binary variable to further segment the family-related travel behaviour.

Categorical Encoding: Eight categorical variables were encoded with the help of LabelEncoder to make the mathematical calculation process easier. The variables were hotel, meal, country, market segment, distribution channel, deposit type, and type of customer.

Temporal Normalization: The arrival date month attribute was codified out of text based names in the form of numbers (1-12) so as to keep the models in a chronological order.

Standardization Protocols: StandardScaler was introduced to offer the identical feature scaling. The scaler was applied to the training data only and the subsequent transformations were only done to the validation and test sets.

Quality Verification: Final validation check was done to ensure that the prepared dataset has zero null value, such that there is good hand over of the prepared dataset to the training framework.

## **II. Model Training & Hyperparameter Selection**

**Lead: Alexia Y. Chavez**

Classifier Implementation: Six different classification algorithms were trained: Logistic Regression, Decision Tree, Random Forest, XGBoost, K-Nearest Neighbors (KNN), and Support Vector Classifier (SVC).

Input Formatting: The pipeline supported the use of raw features in tree-based models (Decision Tree, Random Forest, XGBoost) and a scaling of features to distance-based models (Logistic Regression, KNN, SVC).

Hyperparameter Optimization: The following are certain parameters that have been selected to keep the complexity and generalization of the model under control:

Decision Tree: max depth= 8 to avoid overfitting.

KNN: n/neighbors=11 (odd k) to avoid ties in voting.

SVC: Calibration to probability was turned on to allow ROC-AUC scoring.

Reproducibility: The same random state=42 was used in all the models, to ensure that the output of the various models are consistent across different environments where the models are used.

### **III. Validation, Metrics & Visualization**

**Lead: Cherluk Sumdin**

Evaluation Metrics: An evaluate-standardized evaluate-function was developed to calculate Accuracy, Precision, Recall, F1-Score and ROC-AUC on all models on both the validation and test set.

Visual Documentation: A detailed comparison table (8 models x 5 metrics) was created, and ROC curves as well as confusion matrices of the best-performing models were created.

Performance Analysis: The analysis showed that Logistic Regression had the best ROC-AUC (0.7879 validation / 0.7832 test). This was because the feature space was linearly separable following the preprocessing step and the beneficial exploitation of L2 regularization.

### **IV. Ensemble Design & Documentation**

**Lead: Jade Sanchez**

Ensemble Construction: A Voting Classifier was developed with soft voting on the top three estimators (Logistic Regression, Random Forest, and XGBoost), with weights of [3, 2, 1] used to give priority to the best performing estimator.

Baseline Comparison: A Bayesian baseline (GaussianNB) was used to provide a baseline of the performance against which the superior discrimination of the ensemble was determined by ROC comparison plots.

Technical Reporting: 3-page analysis report and a comparison table PDF were written with the problem statement, methodology, and conclusion. This stage was also the last integration of the GitHub repository system.